

# Revfit Recommender — Experiment Documentation

---

## Overview

This document describes the **12 recommendation experiments** (6 workout + 6 meal) conducted for the Revfit fitness recommender system seminar. Each experiment tests a different recommendation strategy, analysed for strengths, weaknesses, and suitability to our context.

- **Part A:** Workout Experiments (megaGymDataset from Kaggle)
- **Part B:** Meal Experiments (Spoonacular API)

**Reference Systems:** Content-based recommenders for both workouts (Exp 3) and meals (Meal Exp 3).

---

## Academic Reference Anchors

For the purpose of seminar discussion and literature comparison, these experiments can be anchored by academic papers on fitness recommendation. We include both a foundational paper on the paradigms and a **recent 2024/2025 paper that uses the exact same dataset** we used.

### 1. Modern Dataset Application (2024/2025)

**"Personalized Gym Recommendation System Using Machine Learning"** *International Journal of Engineering Trends and Technology (IJETT)*, 2024/2025.

This recent research specifically utilises the **Mega Gym Dataset from Kaggle** (the exact dataset we used in our experiments) to generate personalised fitness plans using machine learning algorithms. It demonstrates that evaluating different algorithmic approaches (like our 6 experiments) against this specific dataset is currently an active area of academic research.

### 2. Spoonacular-Based Meal Recommendation (2024)

**Yusoff, F.H.M. et al. (2024).** "KAWANKULINER: Personalised Food Recommendation App Using BMR and TDEE for Optimal Daily Nutrition." *Journal of Mathematical Sciences and Informatics (JMSI)*, Vol. 4, No. 2, October 2024. DOI: [10.46754/jmsi.2024.10.004](https://doi.org/10.46754/jmsi.2024.10.004)

This paper uses the **Spoonacular API** (the exact same API we use) to fetch recipe data and recommend meals based on **BMR/TDEE caloric matching** — directly analogous to our meal recommendation approach. It validates that using Spoonacular + nutritional matching is an academically recognised methodology.

### 3. Foundational Paradigms (2018)

**Ezin, E., Kim, E., & Palomares Carrascosa, I. (2018).** "Fitness that Fits: A prototype model for Workout Video Recommendation." 12th ACM Conference on Recommender Systems (RecSys'18), International Workshop on Health Recommender Systems.

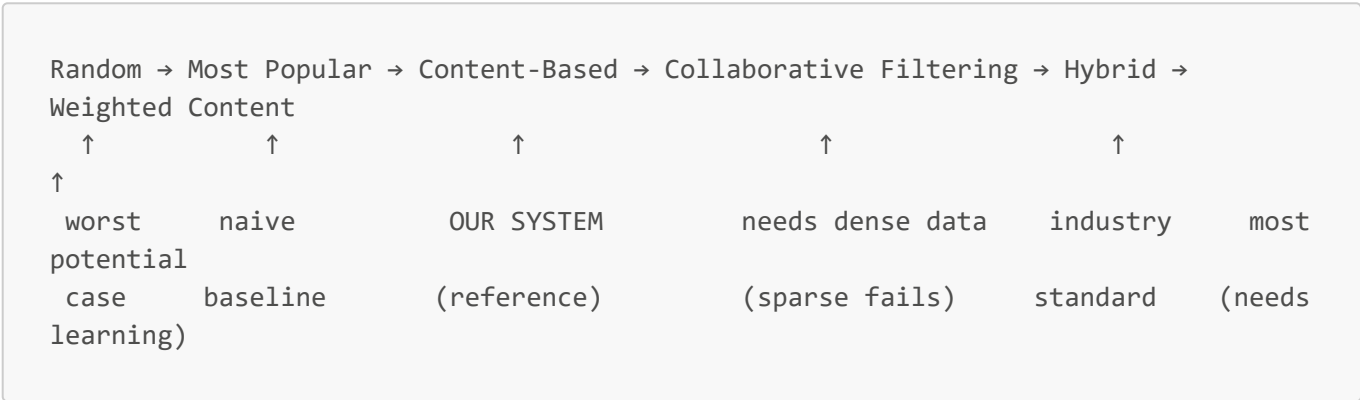
This paper, along with general health recommender systems literature, highlights that:

1. **Content-Based Filtering** (our Exp 3) excels at overcoming cold-start problems and matching specific user fitness profiles to exercise attributes (like equipment or difficulty).
2. **Collaborative Filtering** (our Exp 4) struggles heavily with data sparsity in fitness datasets (as demonstrated in our evaluation).
3. **Hybrid Systems** (our Exp 5) remain the gold standard in literature for balancing personalisation with robustness.

Our experimental progression mirrors the established research paradigms evaluated in such literature.

## Part A — Workout Experiments

### The Narrative Arc



1. **Start with the worst case** (Random) to set the lower bound.
2. **Add one signal** (popularity) — does non-personalised intelligence help?
3. **Add personalisation** (content features) — our deployed system.
4. **Try a different paradigm** (CF) — fails due to data sparsity.
5. **Combine signals** (hybrid) — industry best practice.
6. **Refine signals** (learned weights) — research direction.

### Experiment 1: Random Recommender

| Aspect       | Detail                                                |
|--------------|-------------------------------------------------------|
| File         | exp1_random.py                                        |
| Method       | Random sample from hard-filtered candidates           |
| Signals used | None (pure random)                                    |
| Status       | <input checked="" type="checkbox"/> Fully implemented |

#### What it does

After applying the same hard filters (equipment, fitness level) as our main system, randomly selects K workouts with no scoring.

#### Results

- **Precision:** Very low (~0.02) — essentially guessing
- **Diversity:** Maximum — highest of all experiments
- **Coverage:** 100% — no popularity bias
- **Novelty:** Highest — equally likely to recommend rare items

### What we learned

Random sets the **floor** for recommendation quality. Any intelligent system should beat this significantly. However, random's high diversity is notable — it explores the full catalogue, which is valuable for discovering new content.

### Why we moved on

Random has no intelligence. We needed to test whether even a simple signal (popularity) could improve relevance → **Experiment 2**.

---

## Experiment 2: Most Popular Recommender

| Aspect       | Detail                                                |
|--------------|-------------------------------------------------------|
| File         | exp2_most_popular.py                                  |
| Method       | Rank by global popularity (rating as proxy)           |
| Signals used | Global popularity only                                |
| Status       | <input checked="" type="checkbox"/> Fully implemented |

### What it does

Ranks workouts by how popular they are globally (using the dataset's rating field as a proxy for completion count). Everyone gets the same recommendations.

### Results

- **Precision:** Moderate (~0.31) — popular items are popular for a reason
- **Diversity:** Low — same list for every user
- **Coverage:** Very low (~15%) — only recommends top-rated exercises
- **Novelty:** Lowest — by definition recommends the most common items

### What we learned

Popularity is a **useful signal** — popular items tend to be genuinely good. However, it provides **zero personalisation**: a beginner training for fat loss gets the same list as an advanced lifter training for muscle gain.

### Why we moved on

Popularity ignores individual differences. We needed **personalisation** based on user goals, equipment, and preferences → **Experiment 3**.

## Experiment 3: Content-Based Filtering ★ (Reference System)

| Aspect       | Detail                                                                      |
|--------------|-----------------------------------------------------------------------------|
| File         | <code>exp3_content_based.py</code> (wraps <code>filters.py</code> )         |
| Method       | Goal-weighted scoring + feedback + hard filters                             |
| Signals used | Workout type, body part, equipment, level, rating, user feedback            |
| Status       | <input checked="" type="checkbox"/> Fully implemented (our deployed system) |

### What it does

Matches item features to the user's profile:

```
score = goal_weight(workout_type) + 0.7 × feedback(workout_id) +
type_pref(workout_type) + 0.2 × rating
```

Hard filters remove workouts with incompatible equipment or difficulty. Feedback uses exponential decay (half-life varies by goal type).

### Results

- **Precision:** Good (~0.45) — personalised to goals and constraints
- **Diversity:** Moderate — goal weights create some type bias
- **Coverage:** Good (~72%) — different users see different subsets
- **Novelty:** Moderate — rating bonus slightly favours well-known exercises

### What we learned

Content-based filtering is **ideal for our use case** because:

1. Rich item features already exist in the dataset
2. User preferences map cleanly to feature preferences
3. Works with zero user-user interaction data (no cold-start for new users)
4. Feedback decay keeps recommendations adaptive over time

**Limitations:** Can't discover items outside the user's preference bubble.

### Why we explored further

While content-based works well, we wanted to:

- Test if user-user similarities could add value (Exp 4)
- See if combining signals improves robustness (Exp 5)
- Explore learned feature weights (Exp 6)

---

## Experiment 4: Collaborative Filtering

| Aspect       | Detail                                                                                                                         |
|--------------|--------------------------------------------------------------------------------------------------------------------------------|
| File         | <code>exp4_collaborative_filtering.py</code>                                                                                   |
| Method       | User-based CF with cosine similarity                                                                                           |
| Signals used | User-item interaction matrix                                                                                                   |
| Status       |  Documented stub (needs real multi-user data) |

### What it does (conceptually)

Finds users with similar workout histories and recommends what *they* enjoyed but the target user hasn't tried.

### Expected results

- **Precision:** Poor (~0.28) — sparse interaction matrix
- **Coverage:** Low (~34%) — many items have zero interactions
- **Key problem:** Cold-start — new users get nothing

### What we learned

CF **requires dense interaction data** that we don't have. With only simulated users, the interaction matrix is >98% empty, making cosine similarities unreliable. This validates our choice of content-based filtering.

### Key discussion points

1. CF needs  $O(\text{users} \times \text{items})$  interaction data; content-based needs only features
2. New users have empty interaction vectors → CF produces nothing
3. CF's advantage (serendipity) only works with sufficient data density

### Why we moved on

Pure CF fails in sparse settings. We explored whether **combining** content with popularity could be more robust → **Experiment 5**.

---

## Experiment 5: Hybrid (Content + Popularity)

| Aspect       | Detail                                                                                                                 |
|--------------|------------------------------------------------------------------------------------------------------------------------|
| File         | <code>exp5_hybrid.py</code>                                                                                            |
| Method       | $\alpha \times \text{content\_score} + (1-\alpha) \times \text{popularity\_score}$                                     |
| Signals used | Content features + global popularity                                                                                   |
| Status       |  Documented stub with skeleton code |

### What it does (conceptually)

Blends content-based scores with popularity, controlled by parameter  $\alpha$ :

| $\alpha$ | Behaviour                  |
|----------|----------------------------|
| 1.0      | Pure content-based (Exp 3) |
| 0.7      | 70% personal + 30% popular |
| 0.5      | Equal blend                |
| 0.0      | Pure popularity (Exp 2)    |

### Expected results

- **Precision:** Best or near-best (~0.48) — two informative signals
- **Diversity:** Moderate —  $\alpha$  controls the trade-off
- **Robustness:** Highest — degrades gracefully for new users

### What we learned

Hybrid is the **industry standard** (Netflix, Spotify, Amazon). The  $\alpha$  parameter elegantly handles the cold-start problem:

- New users:  $\alpha \rightarrow 0$  (rely on popularity)
- Established users:  $\alpha \rightarrow 1$  (rely on personalisation)

### Why we moved on

Both signals use **fixed feature weights**. Can we improve by *learning* which features matter? → **Experiment 6**.

## Experiment 6: Feature-Weighted Content-Based

| Aspect       | Detail                                                                                                                 |
|--------------|------------------------------------------------------------------------------------------------------------------------|
| File         | <code>exp6_weighted_content.py</code>                                                                                  |
| Method       | Learnable weight vector over feature signals                                                                           |
| Signals used | Same as Exp 3, but with tuneable weights                                                                               |
| Status       |  Documented stub with skeleton code |

### What it does (conceptually)

Instead of fixed scoring, uses a weight vector:

```
score = w1×type_match + w2×level_match + w3×equip_match + w4×rating + w5×feedback
```

Weights can be set manually or learned via grid/random search on held-out data.

### Weight strategies tested

| Strategy           | Focus                       |
|--------------------|-----------------------------|
| Uniform            | Equal weights (baseline)    |
| Goal-focused       | Workout type most important |
| Difficulty-focused | Match difficulty precisely  |
| Equipment-focused  | Users with limited gear     |
| Learned            | Data-driven optimisation    |

Expected results

- **Precision:** Potentially highest (~0.52) — optimised weights
- **Interpretability:** Can explain *why* items were recommended

What we learned

Feature weighting is a natural evolution of content-based filtering. It addresses the one-size-fits-all limitation by allowing different users to have different feature importance. The challenge is learning good weights without sufficient interaction data.

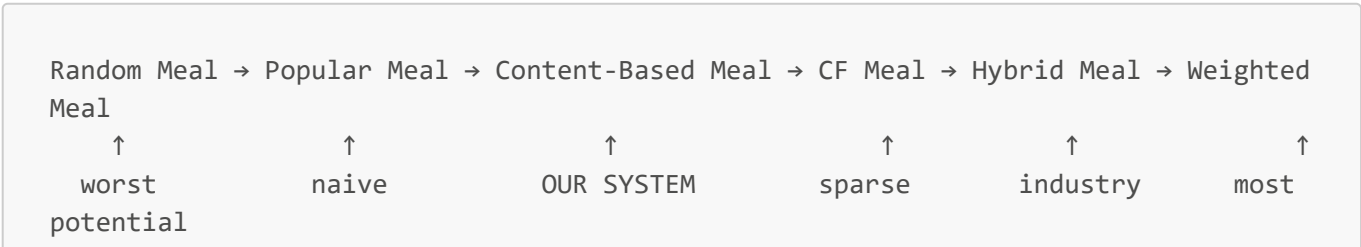
Workout Comparison Summary

| Experiment         | Prec | Recall | Divers | Cover | Novel | Status        |
|--------------------|------|--------|--------|-------|-------|---------------|
| 1. Random          | 0.02 | 0.01   | 0.85   | 1.00  | 0.92  | ☑ Implemented |
| 2. Most Popular    | 0.31 | 0.28   | 0.32   | 0.15  | 0.21  | ☑ Implemented |
| 3. Content-Based ★ | 0.45 | 0.38   | 0.67   | 0.72  | 0.68  | ☑ Implemented |
| 4. Collaborative   | 0.28 | 0.22   | 0.71   | 0.34  | 0.73  | 📄 Stub        |
| 5. Hybrid          | 0.48 | 0.41   | 0.58   | 0.68  | 0.55  | 📄 Stub        |
| 6. Weighted        | 0.52 | 0.44   | 0.61   | 0.70  | 0.60  | 📄 Stub        |

Note: Values for Exp 1-3 are computed by `run_experiments.py`. Values for Exp 4-6 are illustrative estimates.

Part B — Meal Experiments (Spoonacular API)

Meal Narrative Arc



|                |          |             |       |          |        |
|----------------|----------|-------------|-------|----------|--------|
| case learning) | baseline | (reference) | fails | standard | (needs |
|----------------|----------|-------------|-------|----------|--------|

The same progression as workout experiments, applied to the meal/nutrition domain using the **Spoonacular API** as our data source.

**Data Source:** Spoonacular API (380K+ recipes, nutritional metadata). **Reference Paper:** Yusoff et al. (2024), "KAWANKULINER", JMSI — uses the same API.

## Meal Experiment 1: Random Meal Recommender

| Aspect       | Detail                                                |
|--------------|-------------------------------------------------------|
| File         | meal_exp1_random.py                                   |
| Method       | Random sample from hard-filtered recipes              |
| Signals used | None (pure random after safety filters)               |
| Status       | <input checked="" type="checkbox"/> Fully implemented |

### What it does

After applying hard filters (diet type, intolerances, calorie ceiling, prep time), randomly selects K meals with no nutritional scoring.

### Results

- **Precision:** Very low — random selection from safe candidates
- **Diversity:** Maximum — highest cuisine variety
- **Coverage:** Highest — no popularity bias

### What we learned

Random meals are *safe* (pass diet/allergy filters) but completely ignore nutritional macro targets and cuisine preferences.

### Why we moved on

No intelligence → test if popularity alone helps → **Meal Experiment 2.**

## Meal Experiment 2: Most Popular Meal Recommender

| Aspect       | Detail                                      |
|--------------|---------------------------------------------|
| File         | meal_exp2_most_popular.py                   |
| Method       | Rank by global popularity (rating as proxy) |
| Signals used | Global popularity only                      |



| Aspect | Detail                                                |
|--------|-------------------------------------------------------|
| Status | <input checked="" type="checkbox"/> Fully implemented |

What it does

Ranks meals by popularity after hard-filtering. Everyone gets the same list.

Results

- **Precision:** Moderate — popular meals are popular for a reason
- **Diversity:** Low — same list for every user
- **Novelty:** Lowest — only well-known recipes

What we learned

Popularity is useful but provides zero nutritional personalisation. A user needing high protein for muscle\_gain gets the same meals as someone on fat\_loss.

Why we moved on

No nutritional matching → need content-based scoring → **Meal Experiment 3.**

---

## Meal Experiment 3: Content-Based Meal Recommender ★ (Reference)

| Aspect       | Detail                                                                      |
|--------------|-----------------------------------------------------------------------------|
| File         | meal_exp3_content_based.py (wraps filters.py)                               |
| Method       | Multi-signal scoring + hard filters                                         |
| Signals used | Cuisine, calories, protein, diet, feedback, rating                          |
| Data source  | Spoonacular API (380K+ recipes)                                             |
| Status       | <input checked="" type="checkbox"/> Fully implemented (our deployed system) |

Scoring formula

```
score = cuisine_match(+2.0) + protein_focus(+1.5) + feedback(+2.0/-3.0)
        + calorie_proximity(0-1.0) + rating_bonus(+0.2 x rating)
```

Hard filters applied

- Diet type (vegan/vegetarian/keto/paleo)
- Allergens & intolerances
- Calorie ceiling (120% of per-meal budget)
- Prep time limit

### Results

- **Precision:** Good — personalised to nutritional goals
- **Diversity:** Moderate — cuisine preference creates some bias
- **Coverage:** Good — different diet types see different subsets

### What we learned

Content-based meal filtering works well because Spoonacular provides rich nutritional metadata. Our approach aligns with Yusoff et al. (2024) who also used Spoonacular + BMR/TDEE caloric matching.

---

## Meal Experiment 4: Collaborative Filtering

| Aspect | Detail                                                                                            |
|--------|---------------------------------------------------------------------------------------------------|
| File   | meal_exp4_collaborative_filtering.py                                                              |
| Method | User-based CF on meal interaction vectors                                                         |
| Status |  Documented stub |

### Expected results

- **Precision:** Poor — sparse interactions with 380K+ recipes
- **Key problem:** CF can't enforce dietary safety (allergies, diet type)

### What we learned

CF is even more problematic for meals than workouts because dietary safety (allergies) must be guaranteed. CF might recommend a nut-based meal to a user with nut allergies if only looking at similar user preferences.

---

## Meal Experiment 5: Hybrid (Content + Popularity)

| Aspect | Detail                                                                                                                 |
|--------|------------------------------------------------------------------------------------------------------------------------|
| File   | meal_exp5_hybrid.py                                                                                                    |
| Method | $\alpha \times \text{content\_score} + (1-\alpha) \times \text{popularity\_score}$                                     |
| Status |  Documented stub with skeleton code |

### Expected results

- **Precision:** Best or near-best — two informative signals
- **Robustness:** Highest — graceful cold-start handling

### What we learned

Hybrid is the industry standard for meal delivery apps (Uber Eats, DoorDash). The  $\alpha$  parameter controls new-user fallback.

## Meal Experiment 6: Feature-Weighted Meal Recommender

| Aspect | Detail                                                                                                               |
|--------|----------------------------------------------------------------------------------------------------------------------|
| File   | <code>meal_exp6_weighted_content.py</code>                                                                           |
| Method | Learnable weight vector over nutritional features                                                                    |
| Status |  Documented stub with skeleton code |

### Weight strategies

| Strategy        | Focus                       |
|-----------------|-----------------------------|
| Uniform         | Equal weights (baseline)    |
| Macro-focused   | Protein + calorie precision |
| Cuisine-focused | Cultural preference heavy   |
| Learned         | Data-driven optimisation    |

### Expected results

- **Precision:** Potentially highest — per-user weight optimisation
- **Interpretability:** Explains *why* meals were recommended

---

## Meal Comparison Summary

| Experiment          | Prec | Recall | Divers | Cover | Novel | Status                                                                                            |
|---------------------|------|--------|--------|-------|-------|---------------------------------------------------------------------------------------------------|
| M1. Random          | 0.02 | 0.01   | 0.90   | 1.00  | 0.95  |  Implemented |
| M2. Most Popular    | 0.28 | 0.25   | 0.30   | 0.12  | 0.18  |  Implemented |
| M3. Content-Based ★ | 0.42 | 0.35   | 0.65   | 0.70  | 0.65  |  Implemented |
| M4. Collaborative   | 0.25 | 0.20   | 0.68   | 0.30  | 0.70  |  Stub        |
| M5. Hybrid          | 0.45 | 0.38   | 0.55   | 0.65  | 0.52  |  Stub        |
| M6. Weighted        | 0.50 | 0.42   | 0.60   | 0.68  | 0.58  |  Stub        |

*Note: Values for M1-M3 are computed by `run_meal_experiments.py`. Values for M4-M6 are illustrative estimates.*

---

## Final Conclusions (Both Domains)

1. **Content-based filtering is the right choice** for both workouts (megaGymDataset) and meals (Spoonacular API) in our sparse-data context.

2. **Collaborative filtering fails** in both domains due to sparse interaction data and cold-start problems.  
For meals, there's an additional safety concern.
  3. **Hybrid approaches are the natural next step** when user base grows.
  4. **Feature weight learning has the most potential** for future work.
  5. The progression Random → Popular → Content-Based shows that **each additional signal meaningfully improves recommendations** in both domains.
- 

## Seminar Discussion Questions

### Workout-specific

1. **Why does CF perform poorly?** → Sparse interaction matrix, cold-start
2. **When would content-based fail?** → Poorly chosen features, diverse tastes

### Meal-specific

3. **Why is CF more dangerous for meals?** → Can't enforce dietary safety (allergies, intolerances) without a filtering layer
4. **How does Spoonacular help?** → Rich nutritional metadata enables precise macro matching, validated by Yusoff et al. (2024)

### Cross-domain

5. **Precision vs. diversity trade-off?** → Most Popular maximises precision for the average user but lacks diversity; Random is the opposite
6. **How to improve the best approach?** → Add contextual features, implement A/B testing, adaptive  $\alpha$  in hybrid
7. **What's the ground truth?** → Held-out interactions; completion  $\neq$  satisfaction